Dynamic Memory Allocation

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Scenario

- Write a program that reads and stores data from a file
- The first line is **N** indicating the number of student test scores
- Afterward, **N** lines will follow
- Your task is to compute and print the average (properly formatted)

Sample Run

scores.txt

```
5
90
80
85
70
65
```

Sample Output

```
78.00
```

Discussion

- One important issue we need to consider in our solution is determining when the required information becomes available during the program's execution

- When working with arrays, their size must be specified in advance.

- However, in some cases, the size isn't known until the program is running (i.e., at runtime)

Automatic Memory Allocation /1

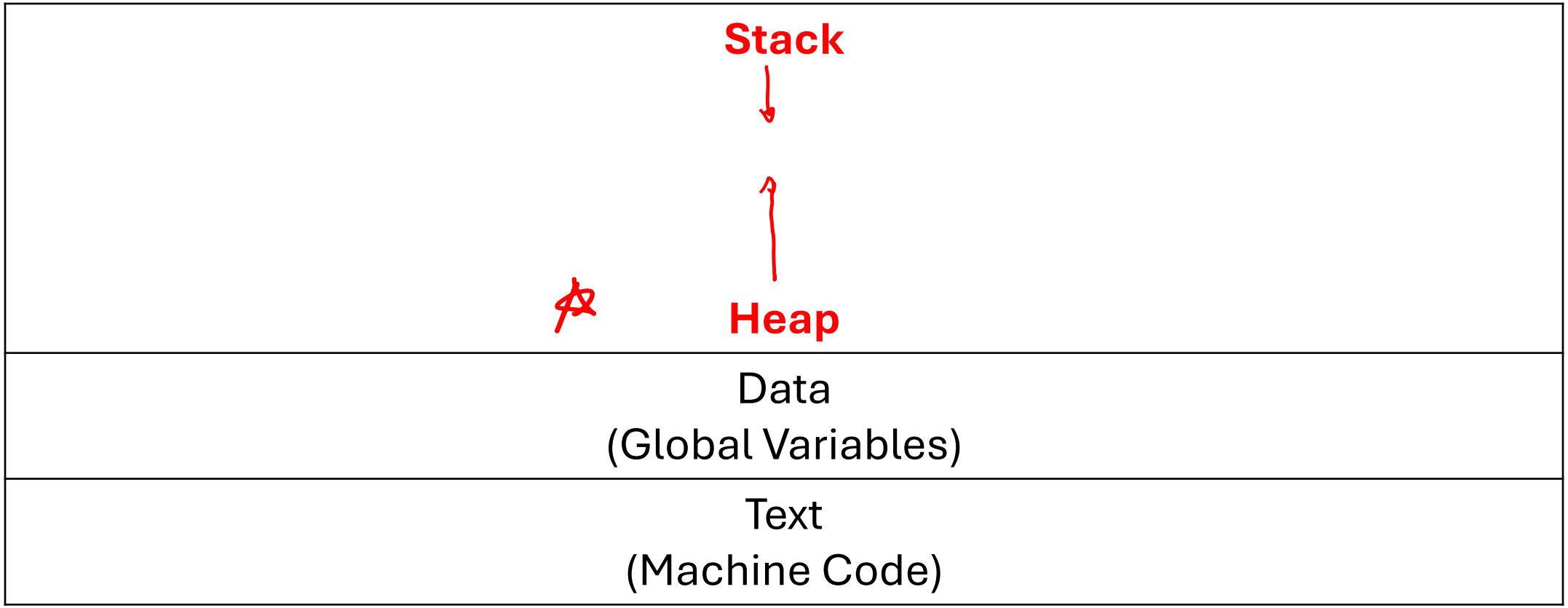
- The **compiler** determined how much memory to allocate for the variables we have declared
- The size of an array **cannot** be changed after it is allocated
- When the program runs, local variables are stored in the **stack space**

Automatic Memory Allocation /2

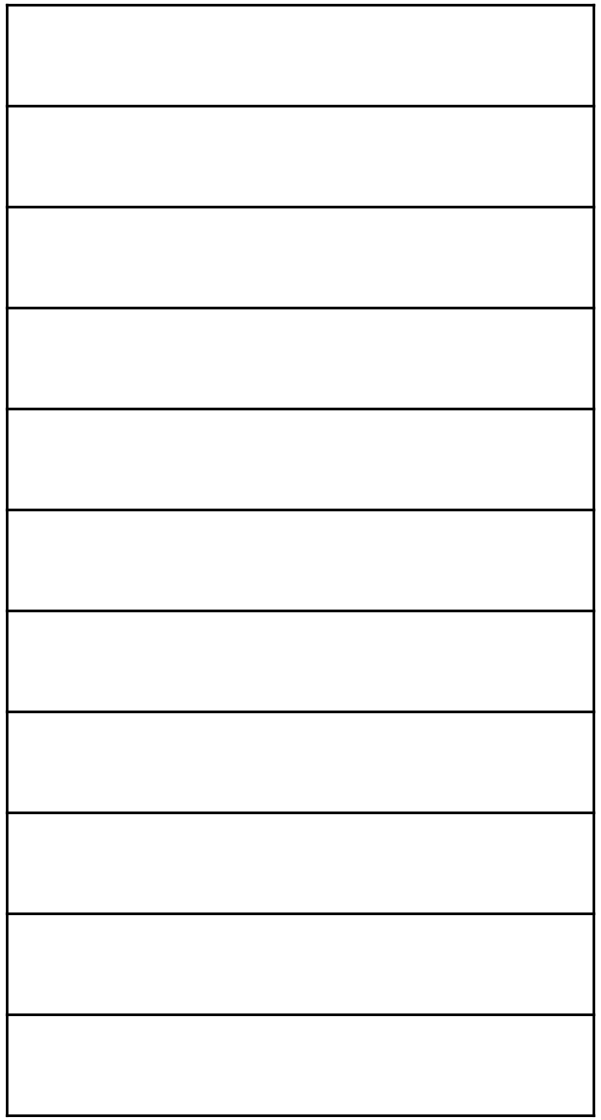
- They stay there during their lifetime (recall: **variable scope**)

- It is **automatically deallocated** at the end of its lifetime

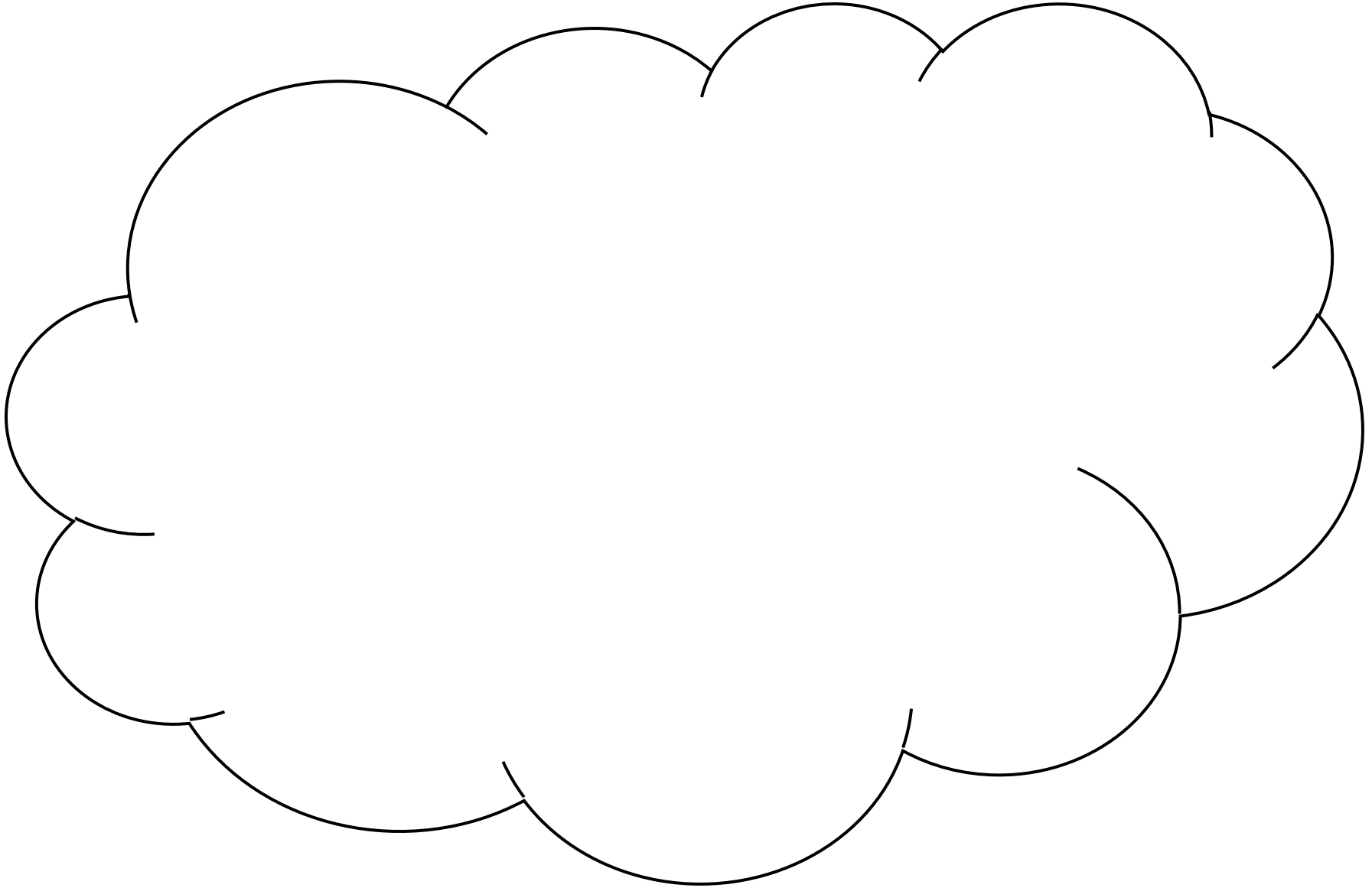
C Program Memory Layout (Segments)



Stack Space



Heap Space



Dynamic Memory Allocation

- Provides flexibility of specifying memory usage size during runtime
- Extra memory that we require will be from the **heap space**
- We simply indicate **how much** space we need
- Hopefully, it is **allocated** at runtime (it’s possible for it to fail)

Best Practices

malloc

stdlib.h

- When you're done with the **dynamic memory**, it needs to be **deallocated** (freed; released)

- Why?

- To ensure that there is no **memory leak** in our program

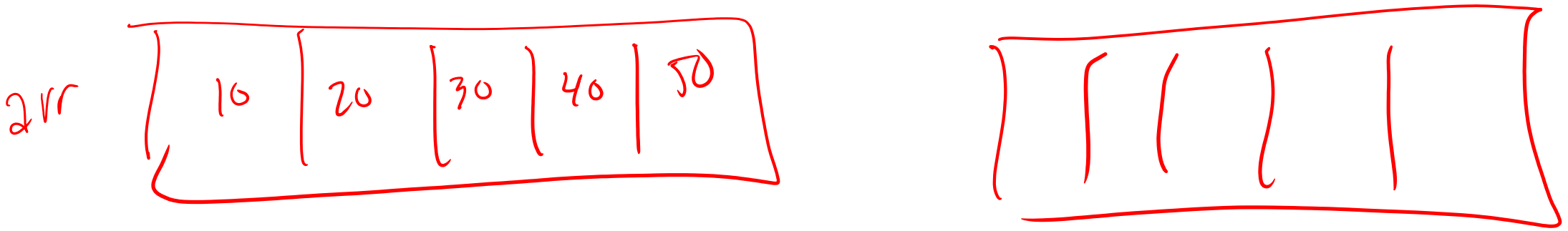
Caution: Memory Leak

- Allocated memory **no longer needed** and was **not released**

- In short, a **`malloc()`** that does not have a corresponding **`free()`**

Practice: Dynamic Arrays

Write a program that prompts the user to enter the size of an array. Then, dynamically allocate memory for an array of **N** integers and initialize all elements to **0**.



Typical Pattern

- Determined the data type of each element
- Identify how many of these you need (i.e., how many elements)
- Call the **`malloc()`** function
- Assign the return value to a pointer variable (data type then **`*`**)

Best Practices

- We will keep in mind the concept of modularity

- Therefore, to reduce the possibility of having a memory leak, we will define **constructor** and **destructor** functions

Practice: Dynamic Arrays

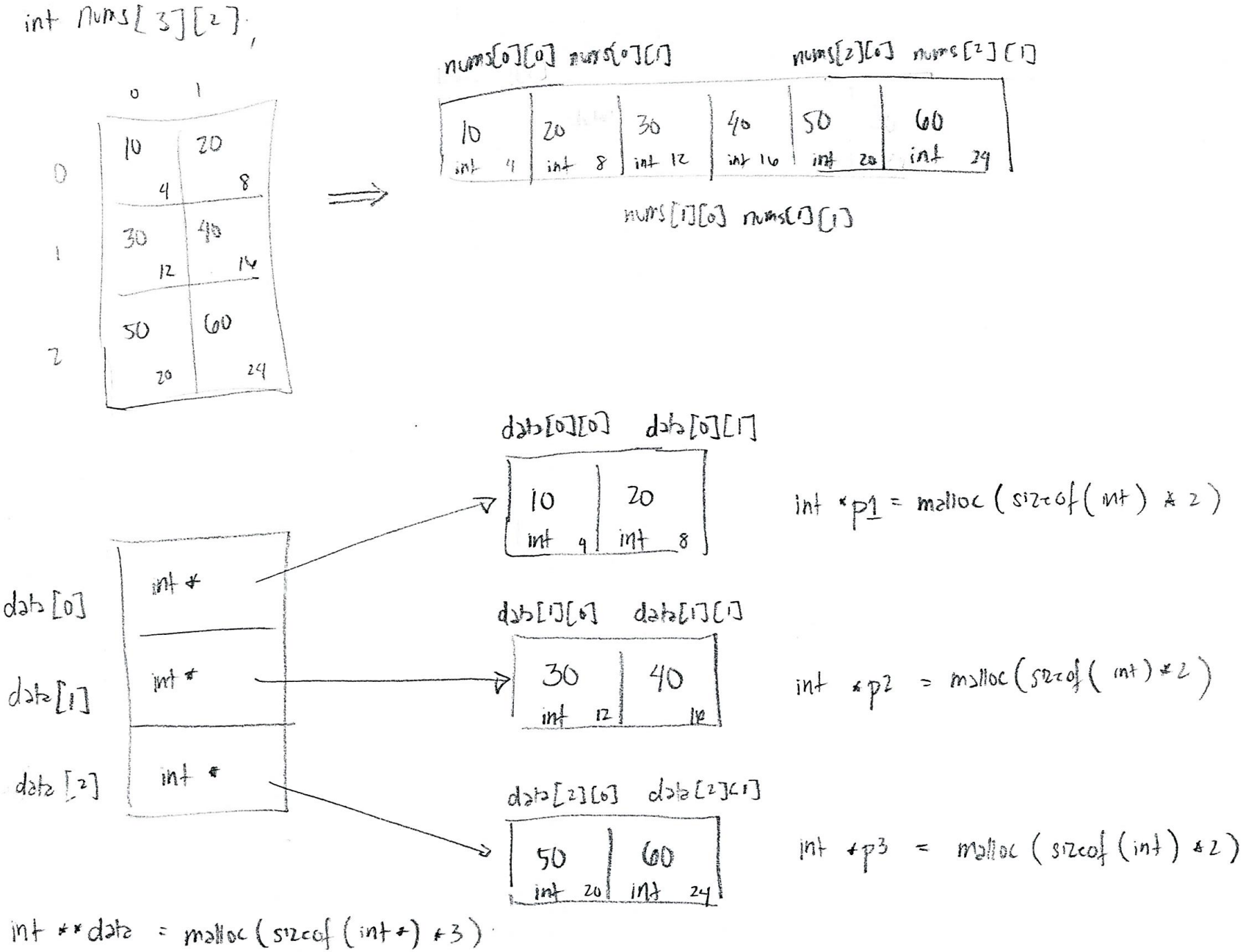
Define two helper functions. The first is **`create_array()`** that takes an integer **`N`** as input and returns a dynamically allocated array of size **`N`**. The second is **`destroy_array()`** that takes a dynamically allocated array as input and frees its memory.

Discussion

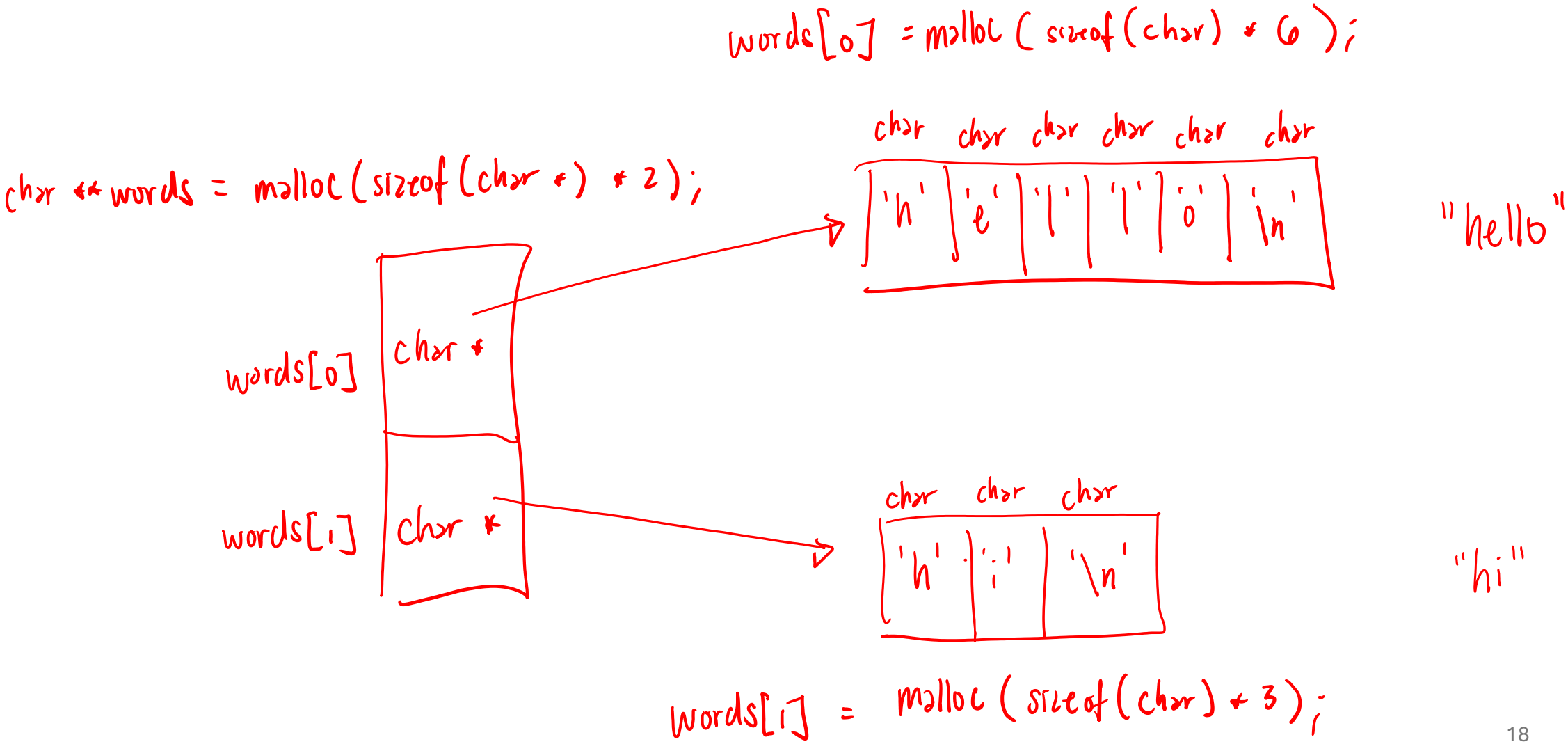
- By now, you should recognize that regardless whether you have a single structure or an array of structures, you are passing the same thing: a pointer

- Treating a pointer as a "dynamic array" is based on context and convention, not on any metadata or type information.

- This means that context matters on how you interpret a function prototype: are you receiving an array or not?


int nums[3][2];
0 1
0
1
2
10 20
4 8
30 40
12 14
50 60
20 24
nums[0][0] nums[0][1] nums[2][0] nums[2][1]
10 20 30 40 50 60
int 4 int 8 int 12 int 16 int 20 int 24
nums[1][0] nums[1][1]
data[0][0] data[0][1]
10 20
int 4 int 8
int *p1 = malloc(sizeof(int) * 2)
data[0]
data[1]
data[2]
int *
int *
int *
data[1][0] data[1][1]
30 40
int 12 int 14
int *p2 = malloc(sizeof(int) * 2)
data[2][0] data[2][1]
50 60
int 20 int 24
int *p3 = malloc(sizeof(int) * 2)
int **data = malloc(sizeof(int*) * 3);

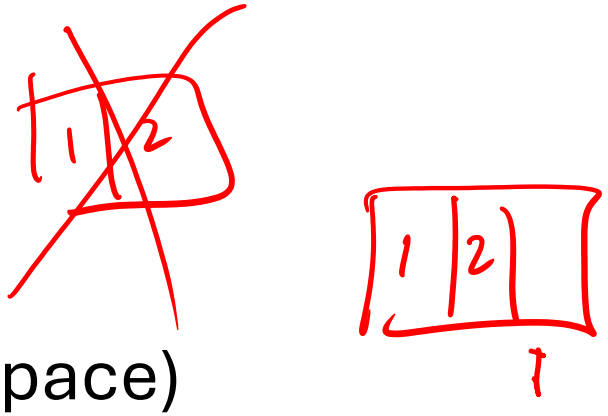
Dynamic Array of Dynamic Strings



Scenario

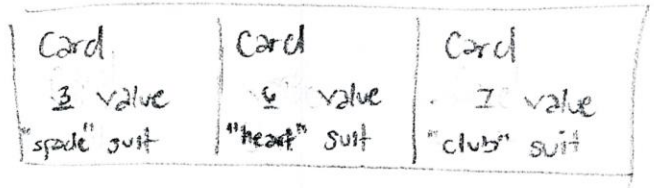
- Say, we want to store a list of numbers (`int`) provided by the user
- Program keeps on asking the user until the user enters `-1`
- The **count** of the numbers in the list is **unknown**
- The memory requirement **cannot be determined** at compile time

Dynamic Array Operations (Functions)



- Insert new value
- Delete value
- Growing the array (double it; if running out of space)
- Shrinking the array (half it; if not fully utilized)
- Search for a value in the array
- Display all values (i.e., contents)
- Sort the values in the array
- Check if array is empty (no valid value stored)

```
Card *p = malloc(sizeof(Card) * 3);   // array of Card
```



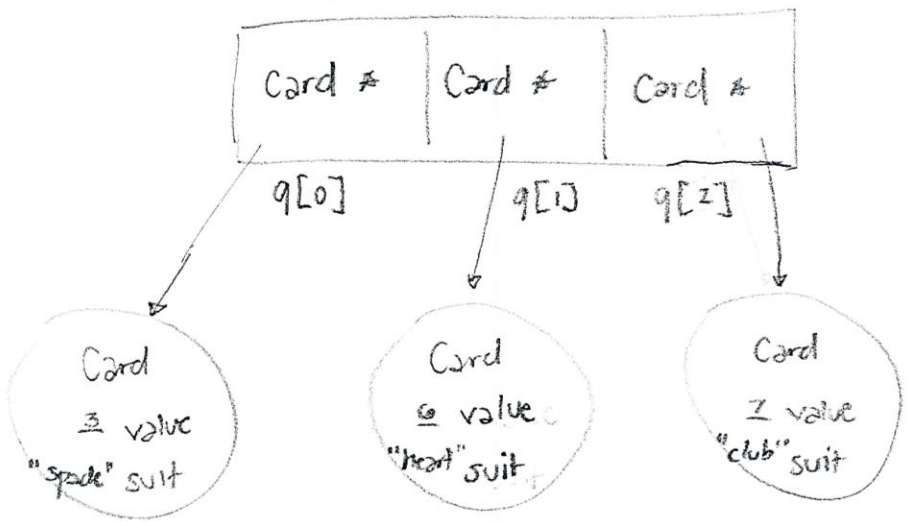
p[0] p[1] p[2]

p[0].value p[1].value p[2].value

p[0].suit p[1].suit p[2].suit

```
typedef struct Card-s {
    int value;
    char suit[101];
} Card;
```

```
Card **q = malloc(sizeof(Card *) * 3);   // array of pointers (to Card)
```

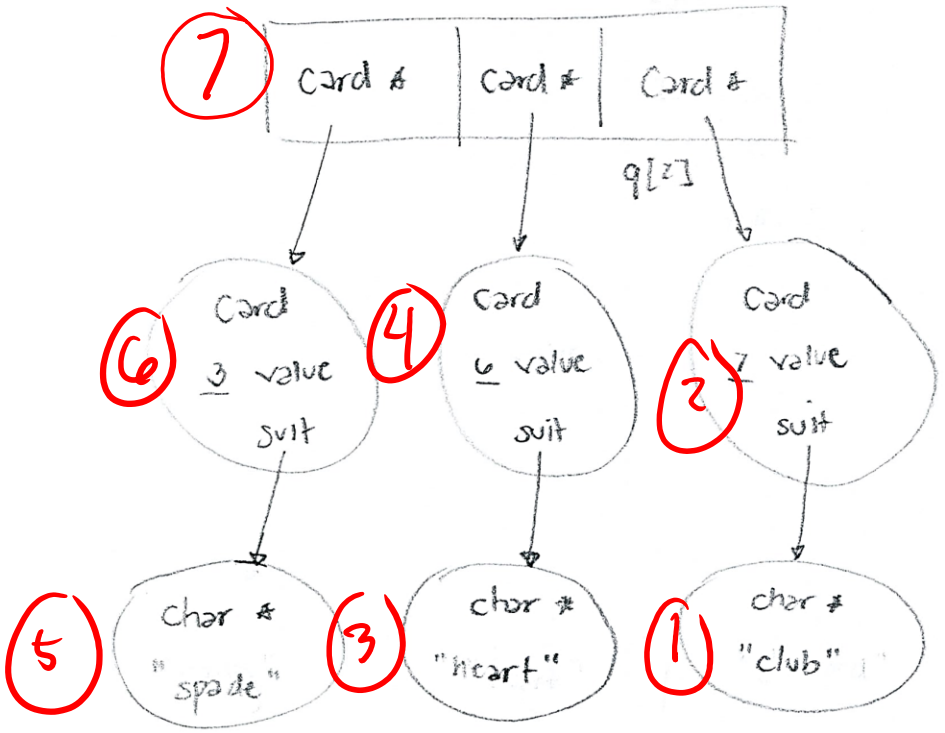



```
q[0] = malloc(sizeof(Card))
q[0] -> value
q[0] -> suit

q[1] = malloc(sizeof(Card))
q[1] -> value
q[1] -> suit

q[2] = malloc(sizeof((Card))
q[2] -> value
q[2] -> suit
```

```
Card **q = malloc(sizeof(Card *) * 3);
```




```
q[2] = malloc(sizeof(card));
```

```
char buf[101];
// assumes buf is a string
q[2]->suit = malloc(sizeof(char) * (strlen(buf)+1));
strcpy(q[2]->suit, buf);
```

```
typedef struct card_s {
    int value;
    char *suit;
} Card;
```

Structures with Dynamic Components

Given the following, how do you set the **`name`** of a **`Person`**?

```
typedef struct Person_s {
    char *name;
    int age;
} Person;
```

Dynamically Allocating Structures

- In this course, the structures we will be dealing with will mostly be dynamically allocated

- Why? We want to pass data efficiently!

Visualizing /1

How do you set the age of the 0-th element?

A dynamically allocated array of structs

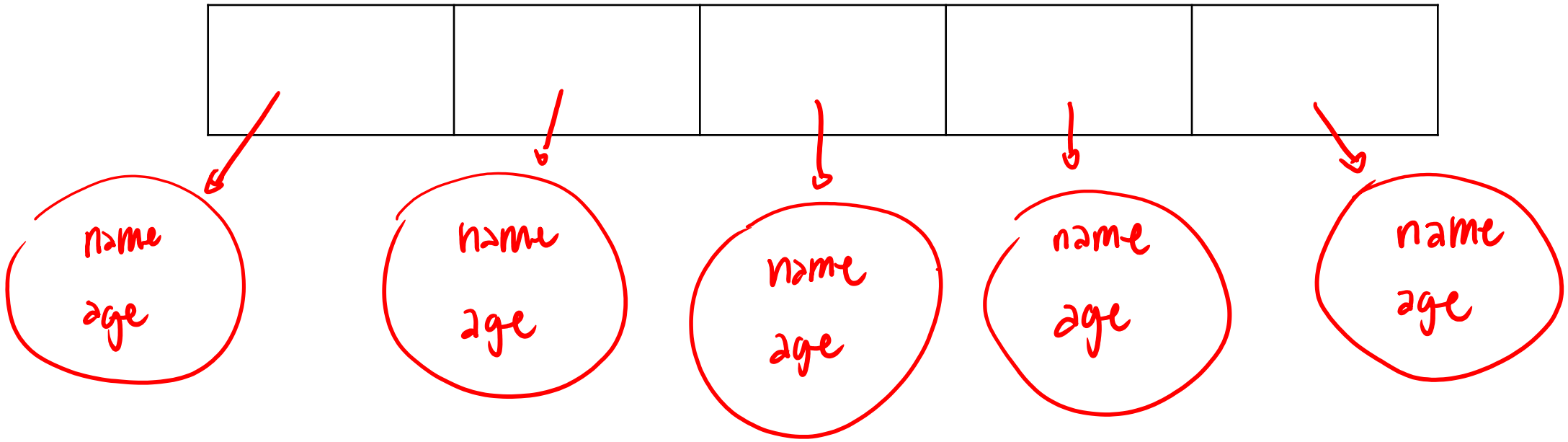
```
Person *p = malloc(sizeof(Person) * 5);
```

Visualizing /2

How do you set the age of the 0-th element?

A dynamically allocated array of pointers to structs

```
Person **p = malloc(sizeof(Person*) * 5);
```



Questions?